

TECHNICAL DEEP DIVE

VeRL-Omni 核心架构解析：多模态强化学习的系统级优化实践

基于技术演讲与配套材料整理

配套材料: verl-omni_slides_20260627.pptx

2026-08-05

原视频： [veRL-Omni：多模态大模型强化学习](#) · **配套资料：** [veRL-Omni Slides](#)

当强化学习的训练对象从纯文本大语言模型扩展到扩散模型与全模态模型，生成和评估两个阶段的计算特征同时发生质变。传统框架围绕“快速自回归采样 + 轻量规则奖励”的假设设计，一旦面对多步去噪轨迹与重度视觉语言奖励，流水线阻塞和资源闲置便接踵而至。VeRL-Omni 正是为解决这一结构性矛盾而从 VeRL 分离出的独立框架 - 它通过异步奖励流、步进式连续批处理和 Rollout 校准三项核心优化，在 Qwen-Image 场景下将单步训练时间从 190 s 压缩至 152 s，端到端吞吐提升约 20%。

本文沿“瓶颈识别 → 架构设计 → 逐层优化 → 算法映射 → 性能验证”的因果链展开，完整剖析这套系统的工程决策与技术权衡。

适读人群： 具备大模型训练基础，希望了解多模态 RL（如扩散模型、全模态模型）系统设计与性能优化的 AI 系统工程师及算法研究员。

前置知识：

- 强化学习基础：PPO / GRPO 算法原理
- 大语言模型分布式训练框架：如 Megatron、FSDP
- 扩散模型（Diffusion Model）基础去噪原理

阅读目标：

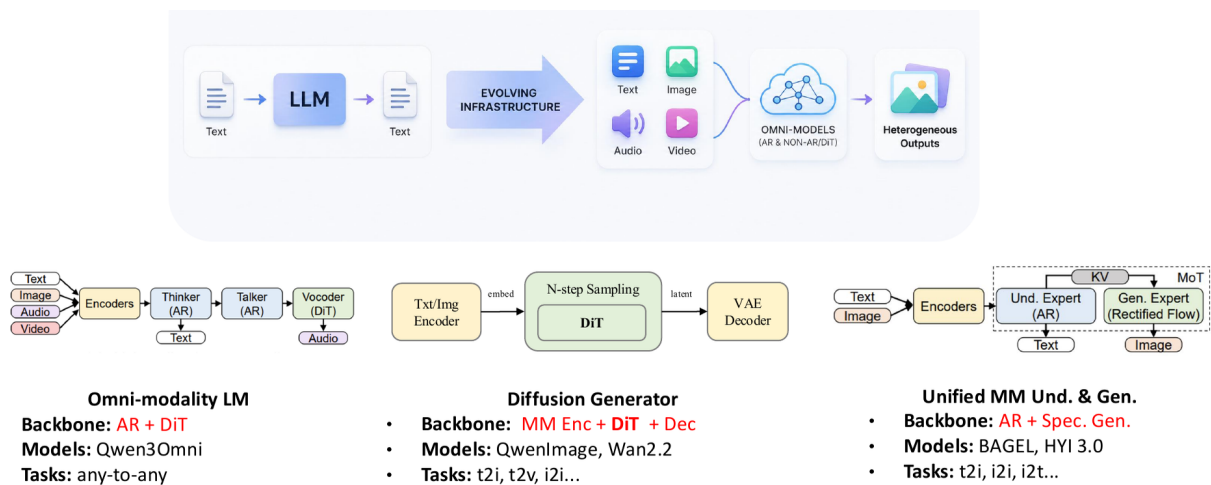
- 1 理解多模态 RL 与传统 LLM RL 在系统设计上的核心差异
- 2 掌握 VeRL-Omni 的 HybridFlow 架构及三大核心优化 - 异步奖励、连续批处理、Rollout 校准
- 3 了解 FlowGRPO 算法在扩散模型中的工程映射与边界条件

一、多模态 RL 的工程矛盾：从 LLM 到扩散模型的范式转移

本节问题： 为什么传统的 LLM RL 框架无法高效支持扩散模型与全模态模型的训练？

模型形态的多样化

从纯文本 LLM 到全模态基础设施，模型架构已经分化出三条截然不同的路线。下图展示了这一演进及三类代表性架构，帮助我们理解多模态 RL 必须面对的异构性。



图注：多模态基础设施的三类架构 - 全模态语言模型、扩散生成器与统一理解-生成模型。来源：演讲 PPT 第 8 页

三条路线的关键特征对比：

架构类型	核心骨干	典型代表	输出模态
全模态语言模型（Omni-modality LM）	AR Thinker + AR Talker + DiT Vocoder	Qwen3-Omni	文本 + 语音
扩散生成器（Diffusion Generator）	多模态编码器 + N 步采样 DiT + VAE 解码器	Qwen-Image, Wan2.2	图像 / 视频
统一理解与生成（Unified MM Und. & Gen.）	AR 理解专家 + Rectified Flow 生成专家	BAGEL	文本 + 图像

关键观察：三类架构均包含非自回归组件 - DiT 声码器、N 步扩散采样或 Rectified Flow。生成过程不再是逐 Token 追加，而是在隐空间（Latent Space，即模型内部的压缩特征空间）中执行多步去噪迭代。这一结构性差异是传统 LLM RL 流水线失效的根本原因。

LLM RL 与 Diffusion RL 的逐项对比

下表将两类 RL 训练场景的关键维度并列对照，是识别系统瓶颈的直接依据。

	LLM RL	Diffusion RL
Use cases	Text generation, reasoning, coding	Instruction-following multimodal generation
Rollout Sampling	AR-token decoding, kv-cache L tokens, $\log P$	Multi-step diffusion on latent space (+SDE stochastic) Img/Vid, T-step trajectory (timesteps, latents, emb, $\log P$)
Sequence length	Varied — long-tail decoding bubbles	Fixed (mostly)
Rollout Backend	vLLM	vLLM-Omni
Algorithm (Advantage)	PPO, GRPO, DPO	FlowGRPO, Diffusion DPO, DiffusionNFT
Reward	Rules, text scorer (low payload)	Preference models, visual/audio VLM (high payload)
Actor (Train)	fixed token sequence $\rightarrow$ per-token $\log P \rightarrow$ single-sequence PG update (one pass)	fixed latent trajectory $\rightarrow$ per-step $\log P \rightarrow$ multi-step PG update (T_{SDE} forwards/backwards)



图注：LLM RL 与 Diffusion RL 在采样、序列长度、后端、算法、奖励及训练更新方式上的差异。来源：演讲 PPT 第 10 页

逐行拆解其中最影响系统设计的维度：

生成采样 (Rollout Sampling)：LLM RL 执行自回归 Token 解码，每步产生一个离散 Token；Diffusion RL 在隐空间中进行多步扩散采样，每步输出一整帧连续隐变量。两者的显存占用模式和算子调度完全不同。

序列长度：LLM 的输出序列长度随问题难度高度可变；扩散模型的去噪步数通常固定（PPT 标注为"Fixed (mostly)"），但每步计算量远大于单次 Token 前向。

采样后端 (Rollout Backend)：LLM RL 普遍使用 vLLM 等高吞吐推理引擎；Diffusion RL 需要承载 DiT (Diffusion Transformer，扩散模型中的 Transformer 骨干网络) 多步推理的专用后端 - VeRL-Omni 中称为 vLLM-Omni。

奖励计算 (Reward)：这是最容易被低估的瓶颈。LLM RL 的奖励往往是规则匹配或轻量文本评分器；Diffusion RL 的奖励却需要调用 VLM (Vision-Language Model，视觉语言模型)、OCR 或偏好模型对生成的图像/音频打分，单次评估的开销可能与一次完整生成相当。

策略更新 (Actor Train)：LLM RL 对单条序列执行策略梯度更新；Diffusion RL 则需要对多步去噪轨迹执行联合更新，算法层面对应 FlowGRPO (Flow-based GRPO，一种适配扩散模型的 GRPO 变体) 等专用算法。

瓶颈的叠加因果链

将以上差异串联，阻塞路径清晰可见：

- 1 **生成变重** - 多步 DiT 采样使单条 Rollout 的耗时和显存成倍增长；
- 2 **评估变重** - VLM/OCR 奖励把"几乎免费"的打分环节变成另一个 GPU 密集任务；

- 3 **流水线停滞** - 传统框架假定奖励计算远快于生成，在同一组 GPU 上串行执行；一旦奖励同样昂贵，训练 Actor 的 GPU 被迫长时间空转；
- 4 **后端不可复用** - vLLM 的 Token 级调度无法直接服务 DiT 推理，强行适配引入额外序列化开销。

四层叠加的结果：即使单卡算力充足，整体训练吞吐仍被最慢环节锁死。

结论：多模态 RL 的核心矛盾在于**生成与评估两端同时变为计算密集型任务**，打破了 LLM RL 框架"轻奖励、快采样"的隐含假设。这是 VeRL-Omni 作为独立仓库从 VeRL 分离、追求更快迭代的直接动因。上述对比主要针对图像与视频扩散场景；对于语音等其他模态，奖励负载特征可能不同，演讲材料未给出量化数据。

二、全局视角：HybridFlow 架构与引擎解耦

本节问题：如何在单一控制器下协调生成、奖励与训练这三个异构且耗时的计算流？

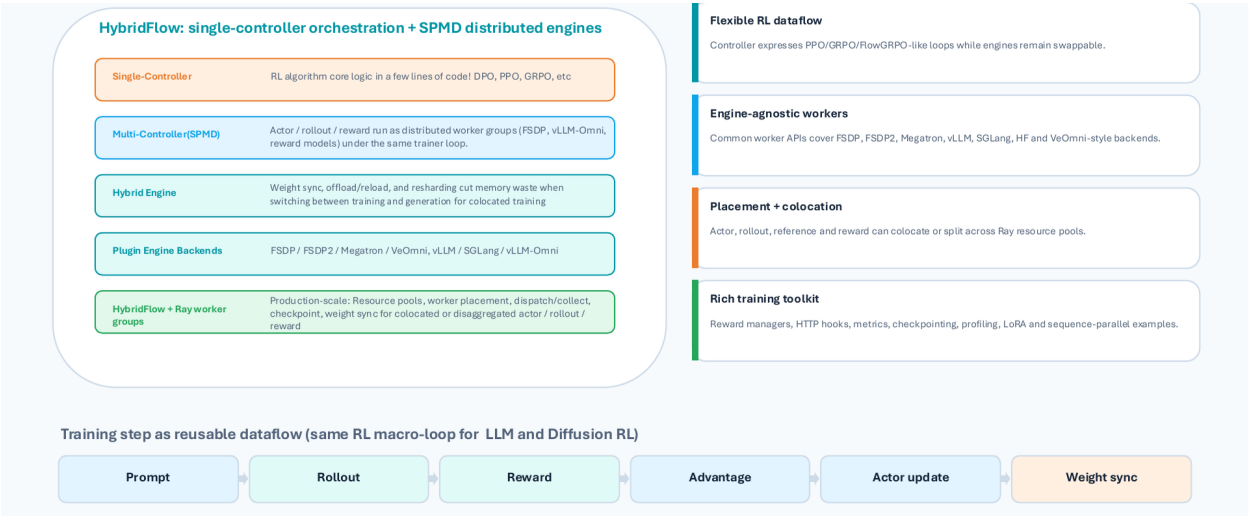
多模态 RL 的一个训练步至少包含三类截然不同的计算：

阶段	计算特征	典型后端
Rollout (生成)	自回归解码或扩散采样，延迟敏感	vLLM-Omni
Reward (奖励)	多模态评估，可异步	视觉/音频奖励模型
Actor Update (训练)	梯度计算与参数更新，显存密集	FSDP2 / VeOmni / Megatron

三者对 GPU 的占用模式、并行策略和显存需求差异极大。VeRL-Omni 的选择是：**用一个单控制器 (Single-Controller) 表达全局 RL 逻辑，同时让每个阶段作为独立的 SPMD (Single Program, Multiple Data, 即同一份程序在多张卡上同时执行的并行模式) 分布式工作组运行** - 这就是 HybridFlow 架构。

架构分层与数据流

下图展示了从控制器到引擎的完整分层结构，帮助理解各组件的职责边界和交互方式。



图注：HybridFlow 分层结构与可复用数据流。来源：演讲 PPT 第 11 页

自上而下呈现四个层次：

Single-Controller（单控制器层）：位于最顶层，负责表达 RL 算法的宏观逻辑 - 例如 FlowGRPO、MixGRPO 或 DPO（Direct Preference Optimization，直接偏好优化）的循环流程。控制器本身是一段普通 Python 控制流，不承担任何张量计算，只发出"执行 Rollout""计算 Reward""更新 Actor"等指令。

SPMD 工作组层：Actor、Rollout、Reward 各自组成独立的分布式工作组（Worker Group）。每个工作组内部通过集合通信（如 NCCL AllReduce）同步，工作组之间则由上层控制器调度串联。

引擎无关层（Engine-Agnostic Workers）：工作组不绑定特定后端。训练引擎可选 FSDP2、VeOmni 或 Megatron；推理引擎可选 vLLM-Omni 或 SGLang。更换硬件或升级框架时只需替换引擎插件，控制器逻辑无需修改。

Ray 资源池：所有工作组运行在 Ray 集群上，支持灵活的放置与共置策略 - 例如 Actor 训练结束后，其 GPU 可立即被 Rollout 工作组复用。

图底部的线性流展示了一个训练步被抽象为可复用的数据流管线：



无论底层模型是大语言模型还是扩散模型，这条宏观循环结构保持一致，差异仅体现在各阶段内部的引擎实现。

最小状态演进：一条图像生成 Prompt 的流转

以一条包含参考图片的图像生成 Prompt 为例，观察它在不同工作组间的完整生命周期：

步骤	所在工作组	动作	输出
1)	控制器	从数据集采样 Prompt，分发给 Rollout 组	Prompt batch
2)	Rollout 组（vLLM-Omni）	执行多模态采样，生成图像轨迹	生成结果（Trajectories）

步骤	所在工作组	动作	输出
3)	Reward 组（视觉奖励模型）	对轨迹进行质量评估	每条轨迹的标量奖励
4)	控制器	汇总奖励，计算优势值（Advantage）	Advantage 张量
5)	Actor 组（FSDP2）	基于优势值执行策略梯度更新	新参数
6)	控制器	将新权重同步回 Rollout 组	权重一致

步骤 2)3) 的结果通过 TransferQueue 或 RPC 机制在工作组间传递。步骤 6) 的权重同步方向是 Actor → Rollout，确保下一轮生成使用最新策略。

边界条件

- **进程间通信开销**：工作组解耦带来灵活性，但轨迹数据（尤其是图像或音频）在组间传递时可能成为瓶颈。演讲材料未给出该环节的具体延迟数据。
- **单控制器的串行约束**：当前数据流是线性管线，Rollout 与 Reward 之间默认串行执行。要实现阶段间流水并行，需在控制器层引入异步调度逻辑 - 下一节将讨论的"步进式连续批处理"和后续的"异步奖励流"正是为此而设计。
- **引擎替换的验证成本**：虽然架构层面实现了引擎无关，但每个新后端仍需适配数据格式和通信协议，并非零成本热插拔。

小结：HybridFlow 通过"单控制器编排 + SPMD 工作组执行"的分层设计，将 RL 训练的全局复杂度收敛到一条可复用的线性数据流上，同时保留了对不同模型架构和硬件后端的适配弹性。接下来，我们逐一深入数据流中的三个瓶颈点 - Rollout、Reward 和 Actor Update - 看 VeRL-Omni 如何逐层优化。

三、生成优化：步进式连续批处理破解去噪延迟

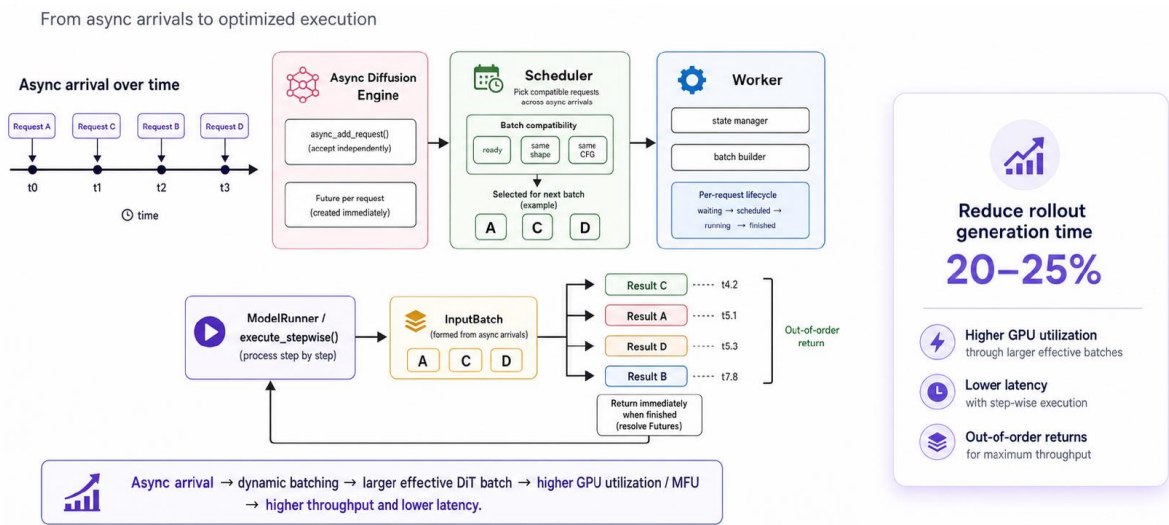
本节问题：扩散模型多步去噪导致请求到达与完成时间不一，如何最大化 Rollout 阶段的 GPU 吞吐？

扩散模型的每条生成请求需经历多步去噪迭代，不同请求的去噪步数可能不同，Latent 形状（即中间隐变量的空间分辨率）也因目标分辨率而异。传统静态批处理把一批请求绑在一起，必须等最慢的那条跑完才能释放 - 步数少的请求被迫陪跑，浪费的算力变成 Padding 开销。

VeRL-Omni 的方案是**步进式连续批处理（Step-wise Continuous Batching）**：以单步去噪为调度粒度，让调度器在每一步重新审视哪些请求可以拼到一起执行，实现请求的动态加入与乱序返回。

执行流水线全貌

下图展示了从异步请求到达、到调度组批、再到乱序返回的完整时间线，是理解该机制的核心。



图注：异步到达的请求经 Scheduler 按形状与 CFG 兼容性组批，由 ModelRunner 逐步执行后乱序返回。来源：演讲 PPT 第 16 页

图中五个关键阶段：

阶段	图中元素	职责
异步到达	时间轴上依次出现的请求 A → C → B → D	不同时刻提交，步数和 Latent 形状各异
异步扩散引擎	Async Diffusion Engine	接收请求并缓存至待调度队列
Scheduler	从队列中挑选兼容子集	按 Latent 形状与是否启用 CFG（Classifier-Free Guidance，无分类器引导）两个维度判断兼容性
Worker / ModelRunner	逐步执行去噪	每完成一步后检查哪些请求已达目标步数
乱序返回	Result C → A → D → B	先完成者先释放显存，腾出槽位给后续请求

为什么能省 20 - 25%：三层因果关系

- 形状对齐消除 Padding：**Scheduler 只把 Latent 形状相同的请求放进同一 Batch，张量维度完全匹配，每一步去噪的有效计算比率接近 100%。
- 步数解耦释放槽位：**已完成去噪的请求立即弹出，空槽在下一步即可被新请求填入，GPU 始终处理有意义的计算。
- 更大的有效 Batch Size：**由于槽位不断回收与补充，DiT 模型每步看到的有效 Batch 通常比静态方案更大，矩阵乘法对硬件计算单元的利用率更高。

演讲 PPT 给出的数据表明，上述机制在 Rollout 生成阶段带来了 **20 - 25%** 的时间缩减。需注意该数字在材料中未附带具体的基线配置或硬件型号等限定条件。

最小例子：四条请求的状态演进

请求	到达顺序	去噪步数	Latent 形状
A	第 1 个	4 步	32×32
C	第 2 个	2 步	32×32
B	第 3 个	5 步	64×64
D	第 4 个	3 步	32×32

- **Step 0 - 1:** A 和 C 先到达，Latent 形状均为 32×32，合批执行前两步。
- **Step 2 结束:** C 仅需 2 步，率先完成返回。D 到达并排队。
- **Step 3:** D (32×32) 填入空槽，与 A 一起执行。B (64×64) 形状不兼容，需等待独立批次。
- **Step 4 结束:** A 完成返回。D 执行最后一步后也返回。
- **Step 5 结束:** B 完成返回。

最终返回顺序 C → A → D → B 与提交顺序 A → C → B → D 完全不同 - 32×32 槽位几乎没有空转。

边界条件

- **收益最大化场景:** 请求间步数差异越大、到达越分散，空闲槽位回收越频繁，吞吐提升越显著。
- **收益退化场景:** 所有请求步数和形状完全一致且同时到达时，机制退化为普通静态批处理，额外调度开销可忽略。
- **形状碎片化风险:** Latent 形状种类过多时，每种能凑出的 Batch 较小，GPU 利用率可能不升反降。演讲材料未给出针对此情形的应对策略。

生成吞吐提升后，大量样本涌入奖励计算阶段。当 VLM 评估的延迟同样高昂时，还需要另一种异步机制来消除评估阶段的流水线气泡。

四、奖励优化：异步流式计算掩盖 VLM 延迟

本节问题：VLM/OCR 奖励评估的延迟极高，如何避免其阻塞整个训练流水线？

同步模式的空白等待

当框架在整个 Batch 的 Rollout 全部完成后才统一启动奖励计算时，时序如下：

阶段	占用资源	是否可并行
Rollout (全 Batch)	Actor GPU	是，但需等最后一条完成
同步屏障	-	所有样本就绪后才进入下一步
Reward 评估 (全 Batch)	Reward Workers / VLM GPU	Batch 内可并行，但与 Rollout 串行
Policy 更新	Actor GPU	等待全部 Reward 返回

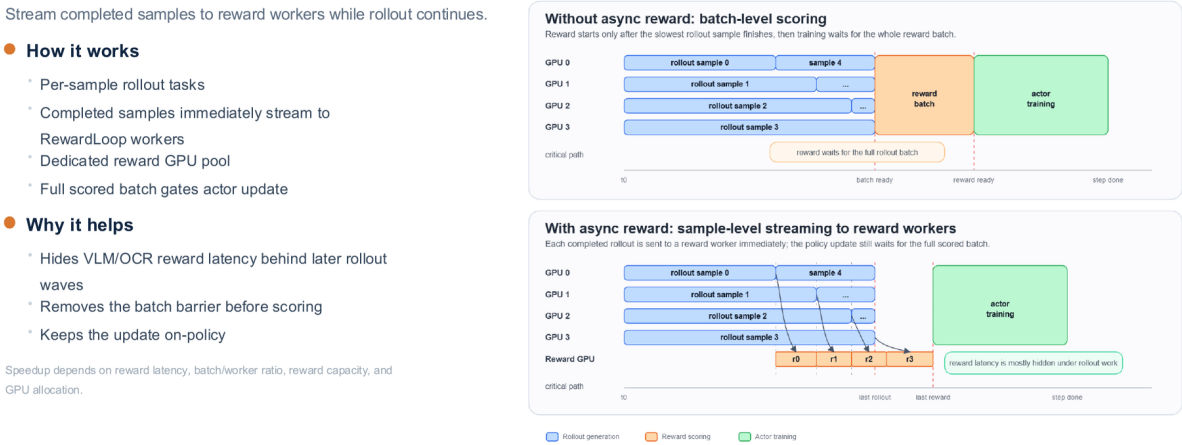
不同样本的生成长度差异很大
一条短回复可能早已完成，却必须等最长的那条结束后才能一同送去打分。评估耗时以整
Batch
的延迟暴露在关键路径上。

异步奖励流的核心机制

VeRL-Omni 打破 Batch 级别的同步屏障，将粒度下沉到单个样本：

- 1 **即时触发** - 任何一条样本生成完毕，立即推送给空闲的 Reward Worker，无需等待同 Batch 其他样本。
- 2 **流水重叠** - 先完成的样本在 Reward Worker 上进行 VLM/OCR 评估时，Actor GPU 继续生成剩余样本。VLM 延迟被"藏"进后续 Rollout 的时间窗口。
- 3 **On-policy 一致性** - 所有样本仍属同一 Rollout Batch，使用同一版本策略生成，更新满足 On-policy 要求。

下图直观对比了两种模式的时序差异，重点观察 Reward 条目的起始时刻。



加速边界

异步流的实际收益高度依赖以下因素：

- **奖励延迟与 Rollout 耗时的比值**：VLM 单条评估越慢且 Rollout 总时间越长，可被隐藏的空间越大。
- **Batch 大小与 Worker 数量的比例**：Worker 不足时样本排队等待，重叠打折。
- **GPU 资源分配**：Actor 与 Reward Worker 共享集群资源，分配失衡导致一方空闲。

演讲材料未给出具体硬件配置下的端到端加速数字，实际收益需根据工作负载和集群规模进行 Profiling。生成与奖励的阻塞问题解决后，Actor 更新阶段又暴露出旧策略对数概率重算的开销 - 这是下一节要处理的瓶颈。

五、训练优化：Rollout 校准与对数概率复用

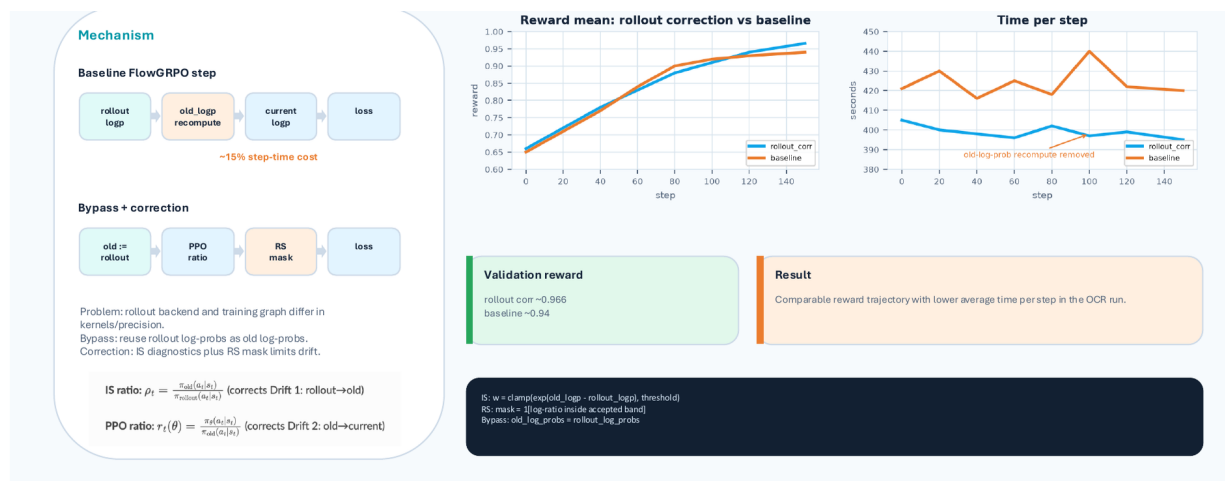
本节问题：生成后端与训练后端的计算图差异，使得重算旧策略对数概率（old logp）开销巨大，如何安全复用生成阶段的数据？

在基于 PPO 的训练中，每步需要两个对数概率：**当前策略对数概率**（current logp）用于计算梯度，**旧策略对数概率**（old logp）用于构建重要性采样比率（Importance Sampling ratio，即新旧策略概率之比，用于校正策略更新幅度）。常规做法是用训练引擎对生成序列做一次额外前向传播来精确计算 old logp。

矛盾在于：VeRL-Omni 的生成后端（vLLM-Omni）与训练后端在算子内核和数值精度上并不完全一致。生成阶段已经产出了每个 token 的对数概率，但直接将其作为 old logp 会引入数学漂移。这形成了一个工程两难：**重算安全但昂贵，复用廉价但有风险。**

基线与 Bypass 流程对比

下图对比了两种策略的流程和实测效果，是判断该优化可行性的直接依据。



图注：左侧为基线和 Bypass 两条流水线的对比；右侧为二者在验证奖励和单步耗时上的实测曲线。来源：演讲 PPT 第

左侧 - 流程对比

步骤	基线 FlowGRPO	Bypass + Correction
1) 获取 rollout logp	生成阶段产出	生成阶段产出
2) 计算 old logp	额外前向传播重算	直接令 $\text{old logp} := \text{rollout logp}$
3) 计算 current logp	训练引擎前向	训练引擎前向
4) 计算 PPO ratio	$\text{ratio} = \exp(\text{current} - \text{old})$	同左，但 old 来自 rollout
5) 异常值处理	PPO clip	RS mask 过滤
6) 计算 Loss	标准 PPO loss	带掩码的 PPO loss

关键差异在步骤 2)：基线方案需要在训练引擎中对完整序列再跑一次前向传播来获得 old logp，这一步约占整个训练步 15% 的时间成本（该数据来自演讲 PPT 第 17 页，测试场景为 FlowGRPO）。Bypass 方案跳过此重算，直接复用生成后端的输出。

右侧 - 验证曲线

在奖励均值维度上，Bypass 方案达到约 0.966，基线约 0.94，处于可比水平（PPT 未给出置信区间，不宜过度解读绝对差异）。在单步耗时维度上，Bypass 曲线明显低于基线，与 15% 的节省幅度一致。

安全复用的因果链

PPO ratio 定义为：

$$r_t = \exp(\log\pi_{\theta}(a_t|s_t) - \log\pi_{\theta_{\text{old}}}(a_t|s_t))$$

其中 $\log\pi_{\theta_{\text{old}}}$ 即 old logp。若 old logp 因内核或精度差异产生偏移 δ ，ratio 被放大为 $r_t \cdot e^{\delta}$ 。当 δ 在长序列上累积时，ratio 会显著偏离 1.0，破坏 PPO 的信赖域约束。

VeRL-Omni 的对策分两层：

- 重要性采样诊断 (IS diagnostics)**：在序列级别计算重要性权重 $w = \prod_t r_t$ ，监测 rollout logp 与训练 logp 之间的系统性偏差。
- 拒绝采样掩码 (Rejection Sampling mask, RS mask)**：对于 w 偏离 1.0 超过阈值的序列，直接将该序列的损失权重置零 - 即丢弃这条样本，而非仅做 ratio 截断。

设计逻辑：**与其在比率空间做截断，不如在样本空间做拒绝。** 截断只能限制单 token 的 ratio 范围，而精度漂移往往在序列维度累积；序列级拒绝能更彻底地隔离异常样本。

最小例子

假设一条 4-token 序列，rollout logp 为 $[-1.20, -0.85, -2.10, -0.50]$ ，训练引擎的 current logp 为 $[-1.18, -0.83, -2.08, -0.49]$ 。

- 基线方案**：训练引擎额外算出 old logp $[-1.19, -0.84, -2.09, -0.50]$ ，与 current logp 差异微小，ratio 接近 1.0。

- **Bypass 方案**：old logp 直接取 rollout 值。每 token 约 0.01 - 0.02 的偏移对 4 token 而言，序列级 w 偏离 1.0 幅度很小，RS mask 不触发，正常参与训练。
若某条 256 token 长序列在每个位置都累积同方向精度偏移，w 可能显著偏离 1.0。此时 RS mask 将其丢弃，阻止漂移传播到梯度更新中。

边界条件

- **RS mask 阈值敏感**：过松则异常 ratio 泄漏，训练不稳定；过紧则大量样本被丢弃，有效 Batch Size 缩水。演讲材料未公开具体阈值数值，部署时需结合 IS 诊断的分布直方图标定。
- **适用范围**：该机制在 PPT 中针对 FlowGRPO 场景展示，对 MixGRPO 等其他算法是否同样适用，材料未明确说明。
- **后端变更需重校准**：不同推理后端的 softmax 内核、浮点累加顺序或量化策略差异均会影响 logp 偏移幅度，更换后端时 RS mask 阈值可能需要重新标定。

至此，系统层面的三项核心优化已逐一剖析。接下来需要回答一个更底层的问题：具体的 RL 算法目标函数是如何在这套系统抽象上表达和执行的？

六、算法映射：FlowGRPO 在扩散模型中的工程实现

本节问题：标准的 GRPO 强化学习目标，如何准确映射到扩散模型的连续去噪过程中？

在 LLM 场景下，GRPO (Group Relative Policy Optimization, 组相对策略优化) 的"动作"是生成一个 token, "轨迹"是完整 token 序列 - 概念清晰且离散。但扩散模型的生成过程是一条连续去噪路径：从纯噪声出发，经过 N 步 SDE (Stochastic Differential Equation, 随机微分方程) 求解产出干净图像。FlowGRPO 通过一套严格的术语映射加两个关键约束，将标准 GRPO 的每个组件对应到这条路径上。

核心概念映射

下图是 FlowGRPO 的完整术语对照表，将标准 RL 概念逐一投射到扩散去噪语境中，是理解后续公式的基础。

Term	Symbol / Code	Meaning in FlowGRPO	One-liner
Sample / episode	1 rollout	One generation: prompt → denoising chain → image	One image + its latent trajectory
State	s_t	Latent z_t at denoising step t, plus conditioning (prompt embeds, timestep)	"What the model sees at this denoising step"
Action	a_t	SDE transition from z_t to the next latent on the chain (not a token, not a pixel)	"One stochastic denoising step"
Trajectory	all_latents	Stored chain z_0, z_1, ... + per-step log-probs from rollout	Full path from noise toward a clean latent
Policy	π_θ	DIT + SDE scheduler: distribution over the next latent given z_t and conditioning	"How the model stochastically denoises"
Probability	P, π(· s_t)	Probability of the actual transition under the policy (Gaussian / SDE kernel)	"How likely this step was"
Log-probability	logP, logp	log π(a_t s_t) for the fixed rollout pair (z_t → z_{t+1})	Stable score for a transition; used in training
Rollout logP	rollout_log_probs	logP from vLLM-Omni at sampling time	"Score from the generation engine"
Old logP	old_log_probs	logP recomputed on the FSDP training graph at step start (anchor); equals rollout in bypass mode	"Proximal anchor for PPO"
Current logP	log_probs	logP from the updating actor on the same transition	"Score used in backprop"
Reward	r	Scalar per image (OCR, VLM judge, preference, etc.)	Outcome reward — usually one score per image
Advantage	A, advantages	Group-normalized reward within the same prompt: (r - μ_group) / σ_group	Good images → positive; bad → negative; broadcast to each denoising step
PPO ratio	r_t(θ)	exp(current_logp - old_logp) = π_θ / π_old	"How much more/less likely this transition is under the current vs. anchor policy"
Clip ratio	ε, clip_ratio	PPO clip band: [1-ε, 1+ε] on the ratio	Limits per-step policy change (FlowGRPO often uses a very small ε, e.g. 1e-4)
Policy loss	FlowGRPO loss	-A · clip(r_t(θ), 1-ε, 1+ε) (mean over steps / samples)	Increase logP on good trajectories, decrease on bad ones
IS ratio (optional)	p_t	π_old / π_rollout (log domain: old - rollout)	Measures rollout-engine vs. training-engine drift
RS mask (optional)	rollout correction	Zero loss weight on steps with extreme log-ratios	Drop unreliable transitions from the gradient

图注：FlowGRPO 术语映射参考表 - 标准 RL 术语、符号、FlowGRPO 含义及直觉。来源：演讲 PPT 第 34 页

三行最关键的映射：

标准 RL 概念	FlowGRPO 对应	直觉
State s_t	第 t 步的带噪潜变量（noisy latent）	去噪路径上的"当前画面"
Action a_t	单步 SDE 去噪转移（stochastic denoising transition）	模型对噪声的一次预测与移除
Trajectory τ	从纯噪声到干净潜变量的完整 N 步路径	一张图像的完整"画法"

与 LLM 中离散 token 采样不同，这里每步动作是一个连续高斯转移策略网络输出去噪均值，而非离散分布上的 logit。

四步执行流程

1) SDE Rollout (轨迹展开)：给定 prompt q，策略扩散模型执行 N 步 SDE 采样，生成 G 条去噪轨迹（G 张候选图像）。每步记录对数似然 log p_θ(a_t | s_t)。

2) Group Advantage (组内优势计算)：G 张候选图像送入视觉奖励模型打分，得到标量奖励 r_1, ..., r_G。在同一 prompt 组内归一化：

$$A_i = (r_i - \text{mean}(\{r\})) / (\text{std}(\{r\}))$$

归一化后的 A_i 广播到该轨迹的每一个去噪步
因为扩散模型的奖励只在生成结束后才可获得，中间步骤没有独立的逐步奖励信号。

3) Clipped Update (截断策略更新)：对每个去噪步计算似然比：

$$p_t = (p_\theta(a_t | s_t)) / (p_\theta_{\text{old}}(a_t | s_t))$$

施加与 PPO 相同的截断操作 (clip 范围参数 ϵ 典型值为 0.2)，防止单次更新步幅过大。映射表区分了三种 log-prob: Rollout logP (生成引擎记录)、Old logP (近端锚点) 和 Current logP (当前参数下反向传播所用)，三者在不同阶段冻结或更新。

4) **KL Anchor (逐步 KL 散度锚定)**：损失函数中加入逐步高斯 KL 惩罚项，度量当前策略与冻结参考模型在每步预测均值上的偏离。由于 SDE 转移的条件分布是高斯的，两个高斯之间的 KL 散度有解析形式，无需蒙特卡罗估计。映射表标注 KL 惩罚系数的典型量级约为 1×10^{-4} 。

最小例子：一张高分图像如何驱动更新

假设 prompt 为“一只猫坐在窗台上”，策略模型一次生成 $G=4$ 张候选图像，奖励分别为 $[0.3, 0.9, 0.5, 0.7]$ 。

- 组内均值 = 0.6，标准差 ≈ 0.2236 ；
- 第 2 张图像的归一化优势 $A_2 \approx +1.34$ (最高)，第 1 张 $A_1 \approx -1.34$ (最低)。

$A_2 = +1.34$ 广播到第 2 条轨迹的全部 N 个去噪步。正优势使该轨迹每一步转移的对数概率被提升，模型未来面对类似 prompt 时更倾向沿这条路径行走。反之，第 1 条轨迹的概率被压低。截断机制确保 $p_t \in [1-\epsilon, 1+\epsilon]$ ，KL 锚定限制整体漂移幅度。

边界条件

映射表中将重要性采样比率 (IS ratio) 和拒绝采样掩码 (RS mask) 标记为**可选项**，说明当前实现不强制依赖这两个机制，具体启用条件演讲材料未给出进一步说明。

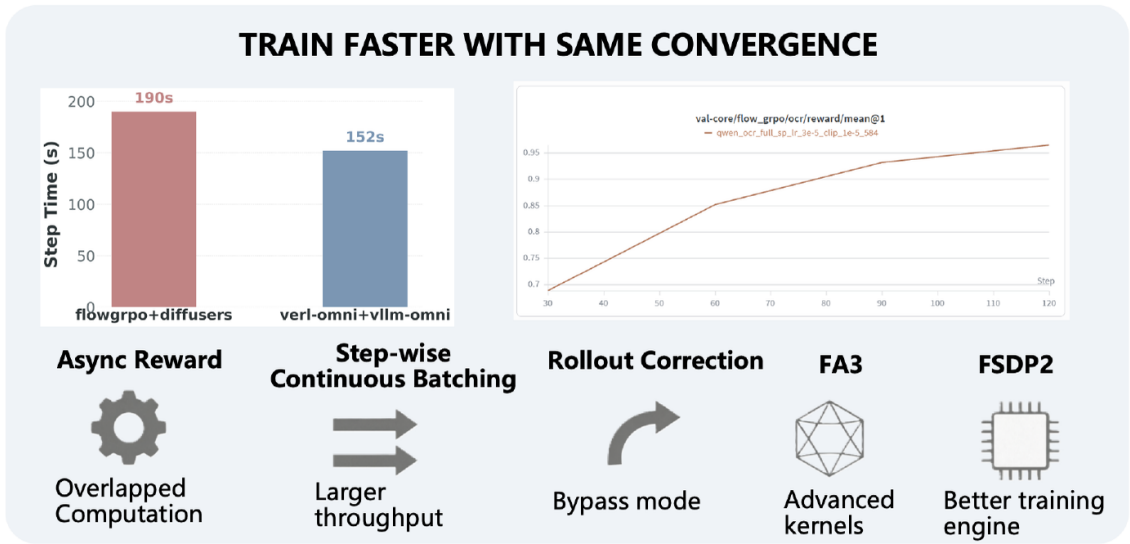
七、端到端性能与硬件泛化：从 GPU 到 NPU

本节问题：上述系统级优化叠加后，实际吞吐收益与硬件适配能力如何？

衡量系统优化是否成功，必须同时满足两个条件：**步时缩短且收敛质量不退化**。

吞吐提升的量化验证

下图给出了在 Qwen-Image 模型上的对比实验，左侧为步时、右侧为收敛曲线，是评估上述优化综合效果的核心数据。



图注：左侧柱状图为单步训练时间对比，右侧折线图为验证集 OCR 奖励均值随训练步数的变化。来源：演讲 PPT 第 5 页

图中关键元素：

元素	含义
红色柱 (190 s)	flowgrpo + diffusers 基线的单步耗时
蓝色柱 (152 s)	verl-omni + vllm-omni 的单步耗时
右侧折线	验证集 OCR 奖励均值 (val-core/flow_grpo/ocr/reward/mean@1)，横轴为训练步数 (30 - 120)
底部五项标注	贡献吞吐提升的五项技术

单步时间由 190 s 降至 152 s，降幅约 20%。这一提升来自五项技术的协同：异步奖励消除流水线气泡、分步连续批处理提高推理利用率、Rollout 校正的 Bypass 模式减少冗余前向传播、FA3 (FlashAttention 3) 提供更高效的注意力计算内核、FSDP2 改善训练阶段的通信与显存效率。

算法维度的收敛观察

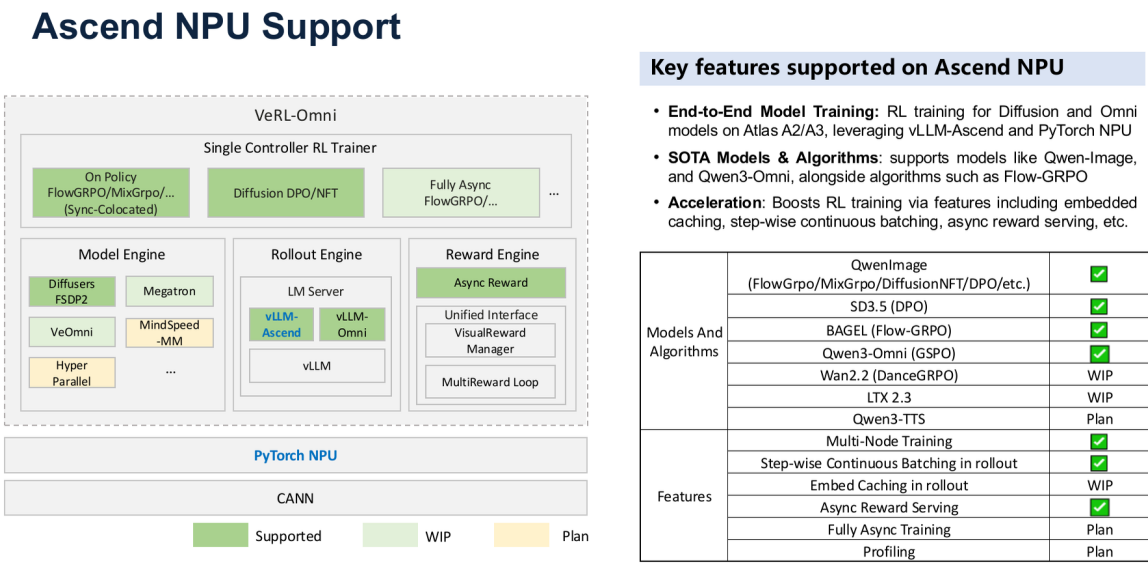
上图右侧只有一条折线，记录的是 FlowGRPO 在 VeRL-Omni 上的验证集 OCR 奖励均值：从第 30 步的约 0.69 稳定爬升，第 90 步前后越过 0.93，到第 120 步接近 0.96。这条曲线的意义不在于“更快”，而在于**形态与基线一致** - 系统优化的贡献体现在步时缩短，而非改变收敛轨迹，说明约 20% 的吞吐提升没有以牺牲训练质量为代价。

算法层面的收敛差异则由另一组实验给出。演讲 PPT 第 27 页单独展示了 Diffusion NFT 算法的奖励曲线，其奖励均值在约 60 步时即突破 0.9，快于 FlowGRPO 的同期水平。需要强调的是，这是**算法选择带来的收敛效率优势**，与上图的系统优化不是同一个维度，两条曲线也来自不同的实验图表，不应放在同一张图上直接比较。

条件限定：20% 的提升幅度与 Qwen-Image 模型及 `diffusers` 基线强相关，具体硬件配置与模型规模在演讲材料中未明确给出。切换到其他模态或不同模型架构时收益可能不同。Diffusion NFT 的收敛判读基于演讲图表的目视观察，演讲材料未给出与 FlowGRPO 的同轴对照实验，置信度为中等。

硬件泛化：Ascend NPU 的初步支持

VeRL-Omni 已将软件栈向华为 Ascend NPU (Neural Processing Unit, 神经网络处理单元) 延伸。下图展示了分层软件架构和功能支持矩阵。



图注：左侧为 VeRL-Omni 在 Ascend 上的分层软件架构，右侧为模型、算法与加速特性的支持状态表。来源：演讲 PPT 第 22 页

软件栈自上而下：

- 1 **Single Controller RL Trainer** - 统一调度层，与 GPU 版本共享逻辑。
- 2 **Model / Rollout / Reward Engine** - 通过 vLLM-Ascend 和 PyTorch NPU 后端接入 NPU 硬件。
- 3 **CANN** (Compute Architecture for Neural Networks) - 华为底层算子库与驱动层。

功能表以颜色区分三种状态：已支持（Supported）、开发中（WIP）和计划中（Plan）。从中可以看到：

- **端到端训练：**Diffusion 和 Omni 类型模型的完整 RL 训练已标记为已支持。
- **模型覆盖：**Qwen-Image、Qwen3-Omni 等主力模型已适配或处于 WIP 阶段。
- **加速特性：**分步连续批处理、异步奖励服务等关键优化已开始移植，但部分仍为 WIP 或 Plan 状态。

局限性：演讲材料未提供 NPU 上的量化性能数据。当前 Ascend 支持应视为"可跑通端到端流程"，而非"性能对齐 GPU"。

结论与局限

回顾全文的因果主线，从瓶颈识别到架构设计再到逐层优化，可以提炼出以下核心结论：

- 1 **多模态 RL 的核心矛盾在于生成与奖励计算同时变为计算密集型任务**，打破了传统 LLM RL 框架“轻奖励、快采样”的隐含假设，必须通过系统级解耦与异步化来应对。
- 2 **HybridFlow 架构将全局复杂度收敛到一条可复用的线性数据流上**，单控制器编排宏观逻辑，SPMD 工作组独立执行各阶段计算，引擎无关设计保留了后端替换的灵活性。
- 3 **步进式连续批处理将调度粒度细化到单步去噪**，通过形状对齐、步数解耦和槽位动态回收，在 Rollout 阶段实现了 20 - 25% 的生成时间缩减（具体基线条件未在材料中详述）。
- 4 **异步奖励流将同步屏障从 Batch 粒度降低到样本粒度**，使 VLM/OCR 评估延迟与后续 Rollout 在时间维度上重叠，收益取决于奖励延迟与 Rollout 耗时的相对比例及 GPU 资源分配。
- 5 **Rollout 校准的 Bypass 模式跳过 old logp 重算**，在 FlowGRPO 场景下节省约 15% 的步时成本，配合 RS mask 的序列级拒绝机制将精度漂移风险控制在可接受范围内。
- 6 **FlowGRPO 通过“动作 = 单步 SDE 转移、轨迹 = 完整去噪路径”的映射**，将离散 token 级别的 GRPO 算法迁移到连续扩散模型上，截断比率与逐步高斯 KL 共同约束更新幅度。
- 7 **在 Qwen-Image 模型上，上述优化叠加后端到端步时从 190 s 降至 152 s (约 20%)**，同时 FlowGRPO 的收敛曲线与基线保持一致；在另一组独立实验中，Diffusion NFT 算法展现出更快的收敛速度，约 60 步即达 0.9 以上的奖励水平。
- 8 **Ascend NPU 的端到端训练流程已跑通**，但多项高级加速特性仍处于 WIP 或 Plan 阶段，演讲材料未提供 NPU 上的量化性能对比数据。

主要局限：吞吐提升数据均基于特定模型与基线组合（Qwen-Image + `diffusers`-based FlowGRPO），迁移到其他场景时需重新验证；RS mask 阈值的最优设定缺乏公开指导，部署时依赖逐场景标定；NPU 侧的性能成熟度尚待量化验证；形状碎片化等极端调度场景的应对策略尚未明确。这些开放问题为后续的工程迭代和社区贡献留下了明确的方向。